1. Fourier Transform & Spectrum

LABSHEET 4

NAME: Saurabh Sharma

CU240251626

ROLL NO: 53

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

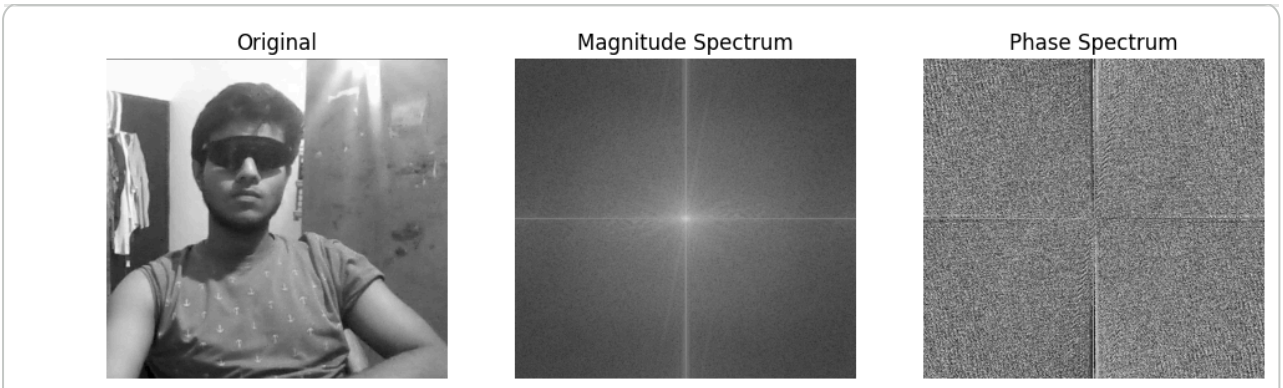
# Using the specified image path
image_path = '/content/Screenshot 2026-06-11 000521.png'
img = cv2.imread(image_path, 0)

F = np.fft.fft2(img)
F_shift = np.fft.fftshift(F)
magnitude = np.log(1 + np.abs(F_shift))
phase = np.angle(F_shift)

plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
plt.imshow(img, cmap='gray')
plt.title("Original")
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(magnitude, cmap='gray')
plt.title("Magnitude Spectrum")
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(phase, cmap='gray')
plt.title("Phase Spectrum")
plt.axis('off')
plt.show()
```



2. Frequency-Domain Low-Pass Filters

```
rows, cols = img.shape
crow, ccol = rows // 2, cols // 2

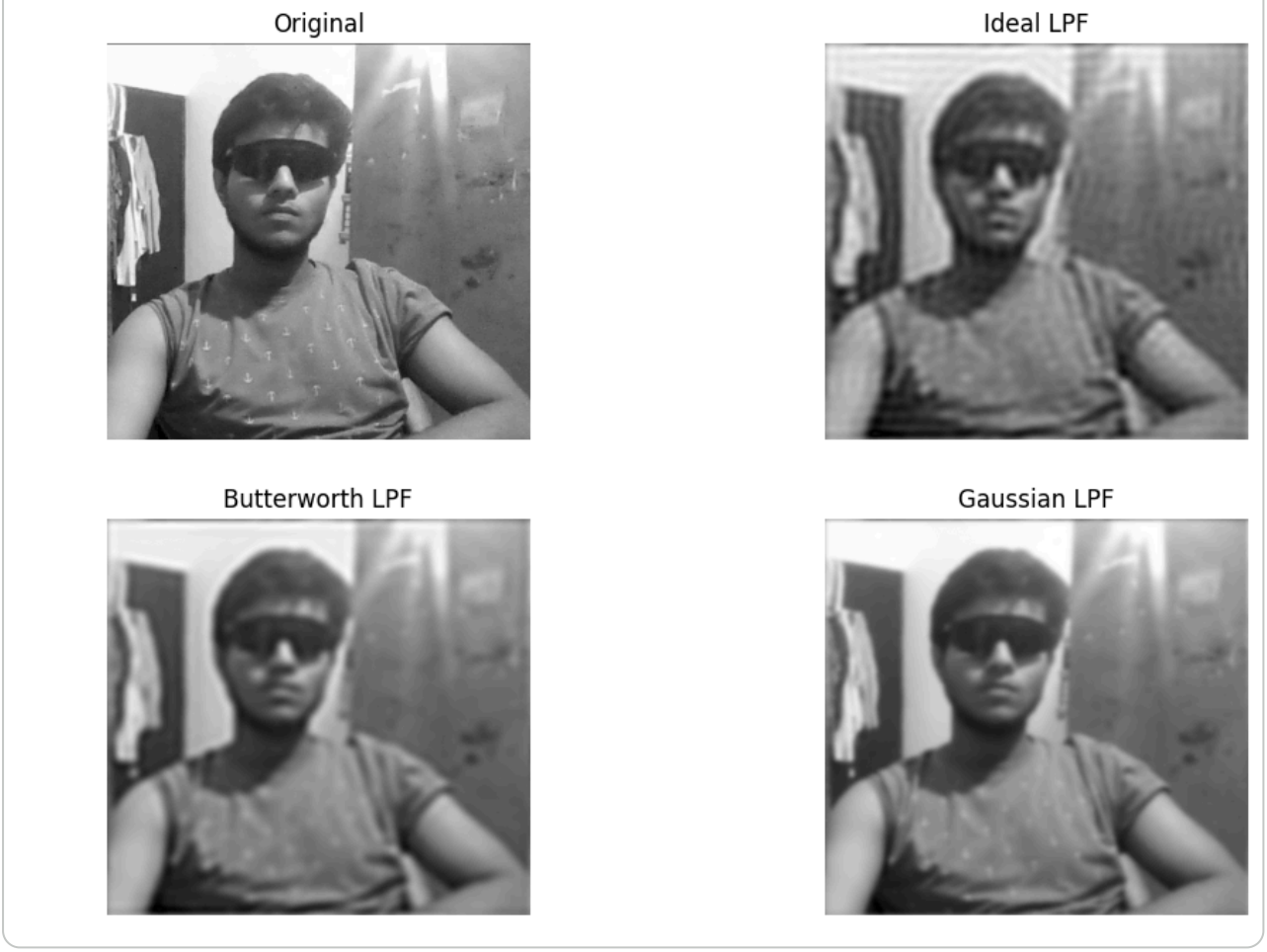
F = np.fft.fftshift(np.fft.fft2(img))
Y, X = np.ogrid[:rows, :cols]
D = np.sqrt((X - ccol)**2 + (Y - crow)**2)
D0 = 30

ideal = (D <= D0).astype(float)
butter = 1 / (1 + (D / D0)**(2 * 4))
gaussian = np.exp(-(D**2) / (2 * D0**2))

def apply_filter(H):
    result = np.fft.ifft2(np.fft.ifftshift(F * H))
    return np.abs(result)

results = [apply_filter(ideal), apply_filter(butter), apply_filter(gaussian)

plt.figure(figsize=(12, 8))
titles = ["Original", "Ideal LPF", "Butterworth LPF", "Gaussian LPF"]
for i, im in enumerate([img] + results):
    plt.subplot(2, 2, i + 1)
    plt.imshow(im, cmap='gray')
    plt.title(titles[i])
    plt.axis('off')
plt.show()
```



3. High-Pass Filter for Edge Enhancement

```
F = np.fft.fftshift(np.fft.fft2(img))
rows, cols = img.shape
crow, ccol = rows // 2, cols // 2
Y, X = np.ogrid[:rows, :cols]
D = np.sqrt((X - ccol)**2 + (Y - crow)**2)
D0 = 30

HPF = (D > D0).astype(float)
filtered = np.fft.ifft2(np.fft.ifftshift(F * HPF))
highpass = np.abs(filtered)
edges = cv2.Canny(img, 100, 200)

plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
plt.imshow(img, cmap='gray')
plt.title("Original")
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(highpass, cmap='gray')
plt.title("Fourier High-Pass")
plt.axis('off')

plt.subplot(1, 3, 3)
```

```
plt.imshow(edges, cmap='gray')
plt.title("Spatial Canny")
plt.axis('off')
plt.show()
```



4. Periodic Noise Removal

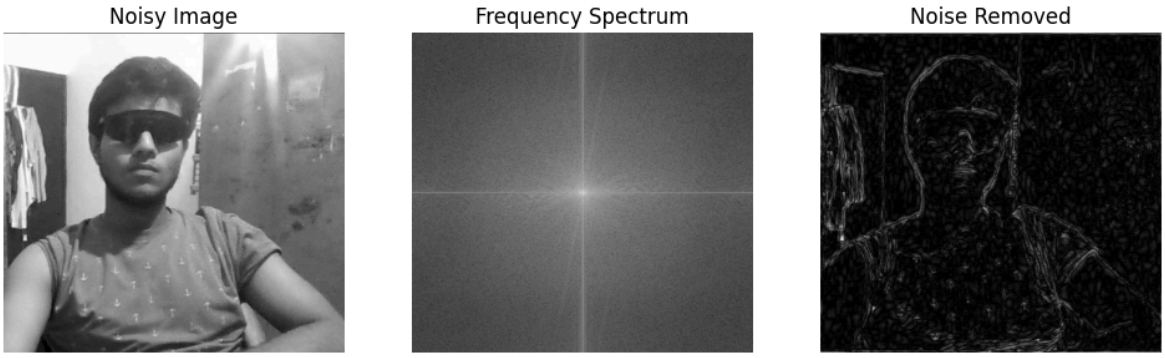
```
F = np.fft.fftshift(np.fft.fft2(img))
spectrum = np.log(1 + np.abs(F))
threshold = np.percentile(np.abs(F), 99.5)
noise_mask = np.abs(F) > threshold

clean_F = F.copy()
clean_F[noise_mask] = 0
clean = np.abs(np.fft.ifft2(np.fft.ifftshift(clean_F)))

plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
plt.imshow(img, cmap='gray')
plt.title("Noisy Image")
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(spectrum, cmap='gray')
plt.title("Frequency Spectrum")
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(clean, cmap='gray')
plt.title("Noise Removed")
plt.axis('off')
plt.show()
```



5. Fourier Compression Using Top-K Coefficients

```
F = np.fft.fft2(img)
K = int(0.05 * F.size)   # Retain 5%
flat = np.abs(F).flatten()
threshold = np.partition(flat, -K)[-K]

F_compressed = np.where(np.abs(F) >= threshold, F, 0)
reconstructed = np.abs(np.fft.ifft2(F_compressed))

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(img, cmap='gray')
plt.title("Original")
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(reconstructed, cmap='gray')
plt.title("5% Fourier Coefficients")
plt.axis('off')
plt.show()

print("Original coefficients:", F.size)
print("Retained coefficients:", np.count_nonzero(F_compressed))
```




```
Original coefficients: 369892
Retained coefficients: 18494
```

6. Fourier-Domain Watermark Hide/Reveal

```
# Create watermark
watermark = np.zeros_like(img)
cv2.putText(watermark, "WATERMARK",
            (50, img.shape[0] // 2),
            cv2.FONT_HERSHEY_SIMPLEX, 2, 255, 5)

F = np.fft.fftshift(np.fft.fft2(img))
W = np.fft.fftshift(np.fft.fft2(watermark))
alpha = 0.01

F_watermarked = F + alpha * W
watermarked = np.abs(np.fft.ifft2(np.fft.ifftshift(F_watermarked)))

# Extract watermark approximately
extracted = np.abs(np.fft.ifft2(np.fft.ifftshift((F_watermarked - F) / alpha

plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
plt.imshow(img, cmap='gray')
plt.title("Original")
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(watermarked, cmap='gray')
plt.title("Watermarked")
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(extracted, cmap='gray')
plt.title("Extracted")
plt.axis('off')
plt.show()
```



7. Band-Pass / Band-Reject Filter

```
F = np.fft.fftshift(np.fft.fft2(img))
rows, cols = img.shape
crow, ccol = rows // 2, cols // 2
```

```
Y, X = np.ogrid[:rows, :cols]
D = np.sqrt((X - ccol)**2 + (Y - crow)**2)

D1, D2 = 20, 60
band_pass = ((D >= D1) & (D <= D2)).astype(float)
band_reject = 1 - band_pass

bp = np.abs(np.fft.ifft2(np.fft.ifftshift(F * band_pass)))
br = np.abs(np.fft.ifft2(np.fft.ifftshift(F * band_reject)))

plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
plt.imshow(img, cmap='gray')
plt.title("Original")
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(bp, cmap='gray')
plt.title("Band-Pass")
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(br, cmap='gray')
plt.title("Band-Reject")
plt.axis('off')
plt.show()
```



8. Swap Magnitude and Phase Between Two Images

(We will use a resized version of our main image alongside the main image itself for demonstration purposes)

```
img1 = img.copy()
# Generating a dummy second image (e.g., flipped) to showcase the swap
img2 = cv2.flip(img1, 1)

F1 = np.fft.fft2(img1)
F2 = np.fft.fft2(img2)

mag1, phase1 = np.abs(F1), np.angle(F1)
mag2, phase2 = np.abs(F2), np.angle(F2)
```

```
result1 = np.abs(np.fft.ifft2(mag1 * np.exp(1j * phase2)))
result2 = np.abs(np.fft.ifft2(mag2 * np.exp(1j * phase1)))

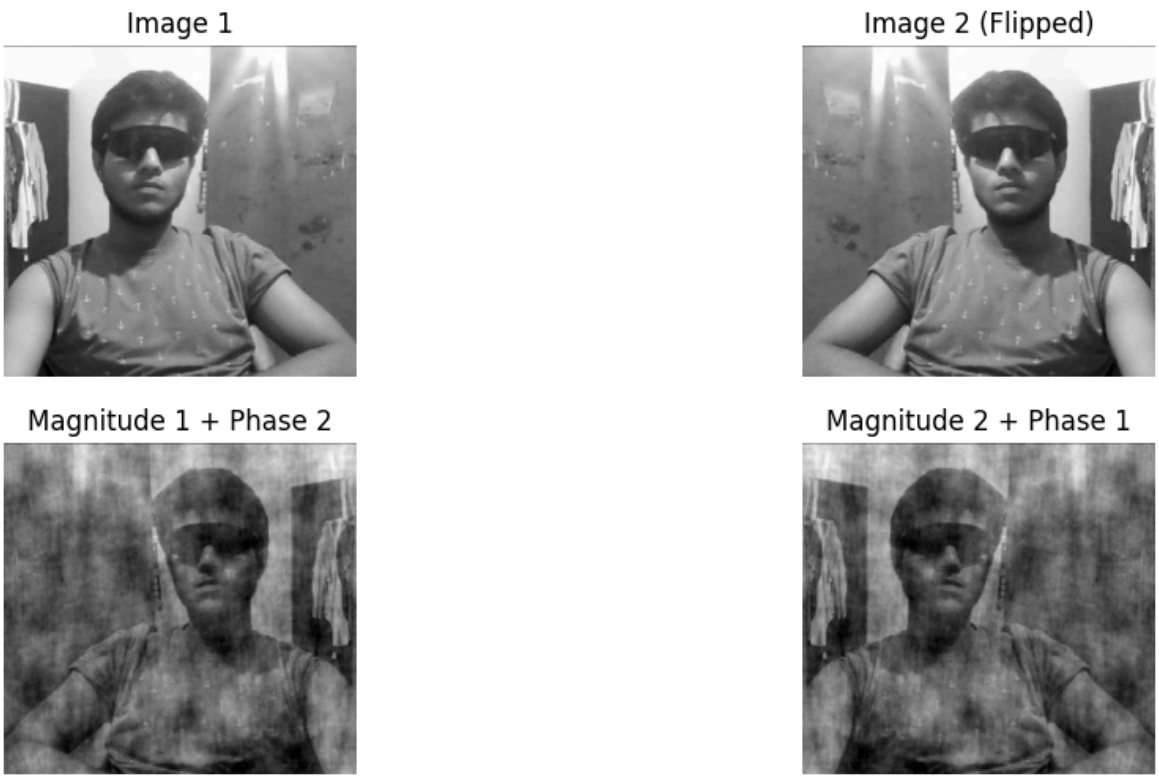
plt.figure(figsize=(12, 6))
plt.subplot(2, 2, 1)
plt.imshow(img1, cmap='gray')
plt.title("Image 1")
plt.axis('off')

plt.subplot(2, 2, 2)
plt.imshow(img2, cmap='gray')
plt.title("Image 2 (Flipped)")
plt.axis('off')

plt.subplot(2, 2, 3)
plt.imshow(result1, cmap='gray')
plt.title("Magnitude 1 + Phase 2")
plt.axis('off')

plt.subplot(2, 2, 4)
plt.imshow(result2, cmap='gray')
plt.title("Magnitude 2 + Phase 1")
plt.axis('off')

plt.show()
```



9. Blur Detection Using High-Frequency Energy

```
F = np.fft.fftshift(np.fft.fft2(img))
rows, cols = img.shape
```

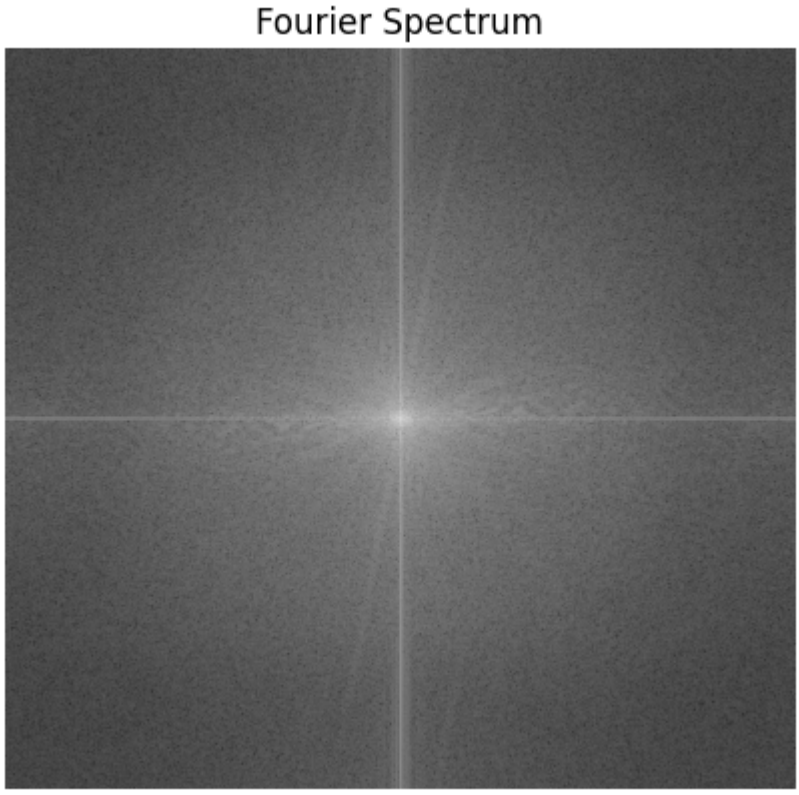
```
crow, ccol = rows // 2, cols // 2
Y, X = np.ogrid[:rows, :cols]
D = np.sqrt((X - ccol)**2 + (Y - crow)**2)

high_freq = np.abs(F)[D > 50]
energy = np.mean(high_freq**2)

print("High-frequency energy:", energy)
if energy < 1e4:
    print("Image is likely BLURRY")
else:
    print("Image is likely SHARP")

plt.imshow(np.log(1 + np.abs(F)), cmap='gray')
plt.title("Fourier Spectrum")
plt.axis('off')
plt.show()
```

```
High-frequency energy: 39808954.4191856
Image is likely SHARP
```



10. Spatial vs Frequency-Domain Filtering

```
# Spatial Gaussian filtering
spatial = cv2.GaussianBlur(img, (11, 11), 0)

# Frequency-domain Gaussian filtering
F = np.fft.fftshift(np.fft.fft2(img))
rows, cols = img.shape
crow, ccol = rows // 2, cols // 2
Y, X = np.ogrid[:rows, :cols]
```

```
D2 = (X - ccol)**2 + (Y - crow)**2
D0 = 30
H = np.exp(-D2 / (2 * D0**2))

frequency = np.abs(np.fft.ifft2(np.fft.ifftshift(F * H)))

plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
plt.imshow(img, cmap='gray')
plt.title("Original")
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(spatial, cmap='gray')
plt.title("Spatial Gaussian")
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(frequency, cmap='gray')
```